Google

Build for everyone

System Design Interview

System Design - Questions are used to assess a candidate's ability to combine knowledge, theory, experience and judgment toward solving a real-world engineering problem. In other words, can a candidate "design policies, processes, procedures, methods, tests, and/or components from the ground up”?

Sample topics include: features sets, interfaces, class hierarchies, distributed systems, designing a system under certain constraints, simplicity, limitations, robustness and tradeoffs. You should also have an understanding of how the internet actually works and be familiar with the various pieces (routers, domain name servers, load balancers, firewalls, etc.). Other topics include: traversing graphs, run-time complexity, distributed hash tables, resource estimation with real systems, big-picture product design, translation of an abstract problem into a system, API discussions, binary trees, caches, mapreduce, indices, compilers, networks, and other common topics.

Operating Systems - You should know processes, threads, concurrency issues, locks, mutexes, semaphores, monitors and how they all work. Understand deadlocks and how to avoid them. Know what resources a process and a thread need. Understand how context switching works, how it's initiated by the operating system and underlying hardware. Know about scheduling. Know the fundamentals of modern concurrency constructs.

Preparing for the System Design Interview:

Interview Tips and Expectations:

1. The system design question will ask you to take an abstract question in a formerly unseen problem and present a high level framework to solve the presented problem. In doing so, follow these steps:

- Gather key requirements
- Identify key design elements
- Identify trade-offs of different decisions
- Dive deep into a specific sub-problem and demonstrate what process to use to arrive at a solution

2. Identify the major components of an overall system design and deeply describe at least two of them. Suggest which technologies and systems should be used to solve a relatively common problem (i.e. relational database, document-based database, etc.), and if relevant, share a story about a time you have solved a problem using these technologies.

3. When defining a system, include as many non-functional requirements as possible. For example: performance, latency, legal, privacy, maintainability, capacity, security, geographic, and cross-data-center consideration.

4. Develop a framework you can use to answer most questions.

5. Approach the problem with an open mind: Be mindful that your familiarity (if any) with the problem may make you take shortcuts (even verbal ones when describing your approach); the interviewer may not share the same context as you. If there is a more succinct approach, feel free to describe to the interviewer the route you're taking and the context behind why.

6. Problem Solve: The interviewers are looking for problem solving and approach as much as they are assessing your solution. So, ask clarifying questions about the problem, express ideas verbally, and think out loud. Constantly challenge your own design. Be prepared to justify every decision you make. Practice going through problems with a friend.

7. Listen: Design questions are typically open-ended and the problem may be ambiguous. If the interviewer gives a hint or asks a question while you are solving the problem, listen intently and do not ignore. They are most likely trying to guide you or looking for a particular awareness.

We expect you to lead the design discussion during this interview. You should ask clarifying and probing questions to understand what the interviewer is looking for. We know and understand that design solutions don't come together in 1 hour. Talk through your solution, make adjustments and dive deep.

a) Follow a structured approach to the problem. You can define your own steps to follow while tackling such questions.

b) Read/practice as many SD questions as you can. In addition to all the documents you found, I liked Grokking the System Design Interview on educative.io

Goal
Clarifying ambiguities early in the interview is critical.
Spend time clearly defining the end goals of the system to have a better chance of success.
Make sure you understand the basic requirements and ask clarification questions. Start with the most basic assumptions:

- *What is the goal of the system?*
- *Who are the users of the system? Who is going to use it?*
- *How are they going to use it?*
- *What are the inputs and outputs of the system?*
- *What do they need it for?*

Features of the system and the APIs to interact with the system and NFRs
Describe the feature set that you'll be talking about. Try to define all the features that you think of by their importance to the user. You don't have to get it right on your first attempt, but make sure that you and your interviewer agree.
Ask clarifying questions, such as:

- *Do we want to discuss the end-to-end experience or just the API?*
- *What clients do we want to support (mobile, web, etc)?*
- *Are we focusing on the backend only or are we developing the front-end too?*
- *Will tweets contain photos and videos?*
- *Will there be any push notification for new (or important) tweets?*
- *Will users be able to search tweets?*
- *Do we need to display hot trending topics?*
- *Do we require authentication? Analytics? Integrating with existing systems?*

Decide which NFR should we focus on: consistency, availability, partitioning, scalability, latency (fast retrieval), durability, redundancy of data

Define what APIs are expected from the system. This will not only establish the exact contract expected from the system but will also ensure if we haven't gotten any requirements wrong. Some examples of APIs for our Twitter-like service will be:

- postTweet(user_id, tweet_data, tweet_location, user_location, timestamp, ...)
- generateTimeline(user_id, current_time, user_location, ...)
- markTweetFavorite(user_id, tweet_id, timestamp, ...)

List the NFRs (Non Functional Requirements) expected from the system. Examples:

- Consistency - Have always an accurate reply ? All users see the same data, at the same time
- Availability - System should be always capable of replying ?
- Partitioning - System continuous to function even if there is no communication between nodes ?
- Reliability - What is stored cannot be lost ?

- Low latency - Efficient response time ?
- Analytics and monitoring ?

Back of the envelope - size the system (back of the envelope calculation) - and database design

It is always a good idea to estimate the scale of the system we're going to design. This will also help later when we will be focusing on scaling, partitioning, load balancing and caching.
Do some math in order to figure out the size of the system

- *What's the ratio between the reads and the writes ?*
- *How many customers in the coming year ? In 3 years time ?*
- *What scale is expected from the system (e.g., number of new tweets, number of tweet views, number of timeline generations per sec., etc.)?*
- *How much storage will we need? We will have different storage requirements if users can have photos and videos in their tweets.*
- *What network bandwidth usage are we expecting? This will be key in deciding how we will manage traffic and balance load between servers.*
- Ballpark numbers
- Back of the envelope calculations

Back-of-the-envelope analysis. This essentially means developing an intuition for the performance of different alternate designs, so that you can reject possible designs out-of-hand, or choose which alternatives to consider more carefully.

High-level design
The goal of this is to outline all the important components that your architecture will need.
Sketch your main components and the connections between them. If you do this, very quickly you will be able to get feedback if you are moving in the right direction.

Start with covering the end-to-end process, based on the goals you've established.
Define the major brick that composes the system and the interfaces it has to expose !
Clarifying questions, such as:

- *Are all the user interactions there ?*
- *Have we covered all the external API ?*
- *Do we need any offline processes ?*

Low-level design
This might include detailing different clients, APIs, backend services, offline processes, network architecture, data stores, and how they all come together to meet the requirements.
Dig deeper into two or three major components; interviewer's feedback should always guide us to what parts of the system need further discussion. We should be able to present different approaches, their pros and cons, and explain why we will prefer one approach on the other. Remember there is no single answer; the only important thing is to consider tradeoffs between different options while keeping system constraints in mind.

Questions to ask yourself ?

- *How should we scale the system ?*
    - *Have you considered the scalability of the AppServers ?*
    - *Have you considered the scaling of the Data ?*
        - *Since we will be storing a huge amount of data, how should we partition our data to distribute it to multiple databases? Should we try to store all the data of a user on the same database? What issue could it cause?*
- *Have we considered the redundancy in terms of processing and data ?*
- *Have we handled the partitioning ? How and why in this way ?*
- *Real time or not real time ?*
- *Do I need some caching and where ?*
    - *How much and at which layer should we introduce cache to speed things up?*

Bottlenecks and To be discussed
Try to discuss as many bottlenecks as possible and different approaches to mitigate them.

- *Is there any single point of failure in our system? What are we doing to mitigate it?*
- *Do we have enough replicas of the data so that if we lose a few servers, we can still serve our users?*
- *Similarly, do we have enough copies of different services running such that a few failures will not trigger total system shutdown?*
- *How are we monitoring the performance of our service? Do we get alerts whenever critical components fail or their performance degrades?*

Assumptions
All the constraints that we give ourselves

Role-related design questions

Due to the wide variety of projects we offer at Google, it is important our design questions are relevant to both your background and the role you will be exploring. Be sure to discuss with your recruiter the role you're pursuing and which system design question is best suited to assess your skills. Depending on your area of expertise, one of the following types of design questions will be asked:

Distributed System Design

System design is a general category which means "big enough systems that you have to operate on a high level of abstraction to get the basic architecture down." A distributed system question is related to problems of coordination between autonomous systems while system design can include problems constrained to a single execution.

Distributed systems questions focus on cross-task coordination, communication, synchronization, and failure modes. System design questions are more weighted toward managing the complexity of large bodies of software. You can find some of the Google research around distributed systems here.

Example Distributed System Design: Design a distributed unique ID service.

Coaching and prep videos:
Leadership, System Design, Coding, Program/Product Management

A couple of important notes:

Please prepare to interview in English, many Googlers around the world interview Brazil's candidates and vice-versa, so we cannot guarantee your interview will be in Portuguese.

You'll need a computer with internet access and camera for your technical interview. In your confirmation email you'll receive the link to the Google Doc (a web-based word processor which lets you share your work and collaborate online) that you'll use for the coding questions in your interview.

Google cannot share specific feedback on any of the stages of your candidacy. We will always keep you in the loop about a positive or negative outcome, but we can not share details.